Ejercicios de Airbnb de Mongo

**1. insertOne Básico: Inserta un nuevo listing con un name (e.g., "Cozy Cabin Retreat"), un summary, property_type: "Cabin",
y price: 150.00.**

```
use('sample_airbnb');
db.getCollection('listingsAndReviews').insertOne({
  name: "Cozy Cabin Retreat",
  summary: "A cozy cabin surrounded by nature.",
  property_type: "Cabin",
  price: 150.00
});
```

2. insertOne con Objeto Anidado: Inserta un listing con la estructura address anidada. Incluye country: "Chile" y market: "Santiago".

```
use('sample_airbnb');
db.getCollection('listingsAndReviews').insertOne({
  name: "Casa Vista Santiago",
  summary: "A comfortable house in downtown Santiago.",
  property_type: "House",
  price: 120.00,
  address: {
    country: "Chile",
    market: "Santiago"
  }
});
```

3. insertMany Simple: Inserta los siguientes dos listings en una sola operación, incluyendo al menos los campos name y price:
- **"Studio de Arte Urbano", price 80.00.**
- **"Loft Minimalista", price 110.00.**

```
use('sample_airbnb');
db.getCollection('listingsAndReviews').insertMany([
  { name: "Studio de Arte Urbano", price: 80.00 },
  { name: "Loft Minimalista", price: 110.00 }
]);
```

4. insertMany con Arrays: Inserta un listing que incluya un array amenities con al menos tres elementos (e.g., "Wifi", "Kitchen", "Free parking").

```
use('sample_airbnb');
db.getCollection('listingsAndReviews').insertMany([
  {
```

```
    name: "Apartamento Centro con Amenities",
    summary: "Equipped apartment near the city center.",
    property_type: "Apartment",
    price: 95.00,
    amenities: ["Wifi", "Kitchen", "Free parking"]
  }
]);
```

5. Validación de id: Inserta un listing con un _id que ya exista en la colección. Observa el error y descríbelo (Este ejercicio demuestra la restricción de unicidad del _id).

```
use('sample_airbnb');
db.getCollection('listingsAndReviews').insertOne({
  _id: "10030955",
  name: "Listing Duplicado"
});
```

Error que da: duplicate key error collection: sample_airbnb.listingsAndReviews index: _id_ dup key: { _id: "10030955" }
MongoDB crea un índice único sobre _id en toda colección, por eso no permite dos documentos con el mismo _id.

6. find por País: Consulta y muestra el name y price de todos los listings ubicados en "Brazil".

```
use('sample_airbnb');
db.getCollection('listingsAndReviews').find(
  { "address.country": "Brazil" },
  { name: 1, price: 1}
);
```

7. find con $gt (Mayor que): Encuentra los listings que tienen más de 5 beds, mostrando solo el name y la number_of_reviews.

```
use('sample_airbnb');
db.getCollection('listingsAndReviews').find(
  { beds: { $gt: 5 } },
  { name: 1, number_of_reviews: 1, _id: 0 }
);
```

8. find con $lt (Menor que): Encuentra los listings cuyo price sea menor a $75.00. Proyecta solo el name, price, y property_type.

```
use('sample_airbnb');
db.getCollection('listingsAndReviews').find(
  { price: { $lt: 75.00 } },
```

```
  { name: 1, price: 1, property_type: 1, _id: 0}
);
```

9. find con $and y Objeto Anidado: Encuentra los listings que sean de tipo "Apartment" y estén ubicados en la ciudad (market) de "New York".

```
use('sample_airbnb');
db.getCollection('listingsAndReviews').find({
  $and: [
    { property_type: "Apartment" },
    { "address.market": "New York" }
  ]
});
```

10. find con $in: Encuentra listings cuyo property_type sea "House" o "Condominium".

```
use('sample_airbnb');
db.getCollection('listingsAndReviews').find({
  property_type: { $in: ["House", "Condominium"] }
});
```

11. find con regex: Encuentra listings cuyo name contenga la palabra "Luxury" (sin distinción de mayúsculas/minúsculas).

```
use('sample_airbnb');
db.getCollection('listingsAndReviews').find({
  name: { $regex: "Luxury", $options: "i" }
});
```

12. find en Array (amenities): Encuentra listings que ofrezcan "Heating" como una de sus amenities.

```
use('sample_airbnb');
db.getCollection('listingsAndReviews').find({
  amenities: "Heating"
});
```

13. find y size de Array: Encuentra listings que tengan exactamente 20 amenities.

```
use('sample_airbnb');
db.getCollection('listingsAndReviews').find({
  amenities: { $size: 20 }
});
```

14. find con sort y limit: Muestra los 10 listings más caros, proyectando el name, price, y address.country.

```
use('sample_airbnb');
db.getCollection('listingsAndReviews').find(
  {},
  { name: 1, price: 1, "address.country": 1, _id: 0 }
).sort({ price: -1 }).limit(10);
```

15. Proyección y Ordenamiento: Muestra los name, summary, y description de los listings que no tienen last_review (campo ausente o nulo), y ordénalos por number_of_reviews descendente.

```
use('sample_airbnb');
db.getCollection('listingsAndReviews').find(
  { last_review: null },
  { name: 1, summary: 1, description: 1, _id: 0 }
).sort({ number_of_reviews: -1 });
```

16. updateOne Simple: Actualiza el property_type del listing "Cozy Cabin Retreat" a "Entire home/apt".

```
use('sample_airbnb');
db.getCollection('listingsAndReviews').updateOne(
  { name: "Cozy Cabin Retreat" },
  { $set: { property_type: "Entire home/apt" } }
);
```

17. updateOne con $set y $push: Actualiza el listing que insertaste en el ejercicio 2 y añade la amenity "Smart TV" al array amenities.

```
use('sample_airbnb');
db.getCollection('listingsAndReviews').updateOne(
  { name: "Casa Vista Santiago" },
  {
    $set: {"address.market": "Santiago Centro"},
    $push: { amenities: "Smart TV" }
  }
);
```

18. updateMany con $inc: Incrementa el number_of_reviews en 1 para todos los listings ubicados en "Spain".

```
use('sample_airbnb');
db.getCollection('listingsAndReviews').updateMany(
  { "address.country": "Spain" },
  { $inc: { number_of_reviews: 1 } }
```

```
);
```

19. updateMany con $pull: Elimina la amenity "Cable TV" del array amenities en todos los listings que la contengan.

```
use('sample_airbnb');
db.getCollection('listingsAndReviews').updateMany(
  { amenities: "Cable TV" },
  { $pull: { amenities: "Cable TV" } }
);
```

20. updateOne con upsert: true: Intenta actualizar un listing que no existe (búsqueda por name: "Mansión de Prueba"). Si no existe, insértalo con name: "Mansión de Prueba", price: 1500.00, y country: "United States".

```
use('sample_airbnb');
db.getCollection('listingsAndReviews').updateOne(
  { name: "Mansión de Prueba" },
  {
    $setOnInsert: {
      price: 1500.00,
      "address.country": "United States"
    }
  },
  { upsert: true }
);
```

21. $match y $count: Cuenta cuántos listings tienen más de 10 amenities.

```
use('sample_airbnb');
db.getCollection('listingsAndReviews').aggregate([
  { $match: { "amenities.10": { $exists: true } } },
  { $count: "mas_de_10_amenities" }
]);
```

22. $group para Precio Promedio por País: Calcula el precio promedio de los listings por cada address.country.

```
use('sample_airbnb');
db.getCollection('listingsAndReviews').aggregate([
  {
    $group: {
      _id: "$address.country",
      precio_promedio: { $avg: "$price" }
    }
  }
]);
```

23. $group y $unwind (Top 5 Amenities): Despliega el array amenities, cuenta su frecuencia y muestra las 5 amenities más comunes en todo el dataset.

```
use('sample_airbnb');
db.getCollection('listingsAndReviews').aggregate([
  { $unwind: "$amenities" },
  { $group: { _id: "$amenities", frecuencia: { $sum: 1 } } },
  { $sort: { frecuencia: -1 } },
  { $limit: 5 }
]);
```

24. $match, $group, y $sort: Calcula la cantidad de listings por property_type y ordena el resultado para ver los tipos más comunes.

```
use('sample_airbnb');
db.getCollection('listingsAndReviews').aggregate([
  { $match: { property_type: { $exists: true } } },
  { $group: { _id: "$property_type", cantidad: { $sum: 1 } } },
  { $sort: { cantidad: -1 } }
]);
```

25. $project y Conversión de Tipo: Muestra el name, price, y crea un nuevo campo price_entero con el valor del precio convertido a entero (usando $toInt).

```
use('sample_airbnb');
db.getCollection('listingsAndReviews').aggregate([
  {
    $project: {_id: 0, name: 1, price: 1, price_entero: { $toInt: "$price" }}
  }
]);
```

26. Pipeline Complejo (Análisis de Review Scores): Filtra los listings con al menos 10 number_of_reviews. Luego, calcula el promedio de review_scores.review_scores_rating para ese subconjunto.

```
use('sample_airbnb');
db.getCollection('listingsAndReviews').aggregate([
  { $match: { number_of_reviews: { $gte: 10 } } },
  {
    $group: {
      _id: null,
      promedio_rating: { $avg: "$review_scores.review_scores_rating" }
    }
  }
]);
```

27. $bucket para Rangos de Precio: Agrupa los listings en rangos de precio: 0-100, 101-300, 301-1000, 1000+. Muestra la cantidad de listings en cada rango.

```
use('sample_airbnb');
db.getCollection('listingsAndReviews').aggregate([
  {
    $bucket: {
      groupBy: "$price",
      boundaries: [0, 101, 301, 1001],
      default: "1000+",
      output: { cantidad: { $sum: 1 } }
    }
  }
]);
```

28. $lookup (Simulación de JOIN): (Este ejercicio es teórico, ya que no hay una colección secundaria simple para unir en sample_airbnb). Describe un pipeline que usaría $lookup si existiera una colección hosts para unir listings con la información del anfitrión por host.host_id.

No se ejecuta porque la colección hosts no existe.
Ojo: from = colección a unir, localField = campo en listings, foreignField = campo equivalente en hosts, as = arreglo donde queda la info del anfitrión.

```
use('sample_airbnb');
db.getCollection('listingsAndReviews').aggregate([
  {
    $lookup: {
      from: "hosts",
      localField: "host.host_id",
      foreignField: "host_id",
      as: "info_anfitrion"
    }
  }
]);
```

29. Cálculo de Densidad de Huéspedes: Para cada property_type, calcula el promedio del campo accommodates (capacidad máxima de huéspedes) y ordena de mayor a menor.

```
use('sample_airbnb');
db.getCollection('listingsAndReviews').aggregate([
  {
    $group: {
      _id: "$property_type",
      promedio_accommodates: { $avg: "$accommodates" }
    }
```

```
  },
  { $sort: { promedio_accommodates: -1 } }
]);
```

30. Análisis por Fecha (Últimas Reviews): Filtra los listings que tienen una last_review registrada. Utiliza $group para contar cuántos listings tuvieron su última revisión en cada año. (Usa $year en $group).

```
use('sample_airbnb');
db.getCollection('listingsAndReviews').aggregate([
  { $match: { last_review: { $type: "date" } } },
  {
    $group: {
      _id: { $year: "$last_review" },
      cantidad: { $sum: 1 }
    }
  },
  { $sort: { _id: 1 } }
]);
```